Want to do another round of practice, or do you want me to help you with something specific?

okay list down all the probable questions that might be asked with their textbook definitions. Cover whatever is expected to answer for a question.

Orchestrated comprehensive interview prep across admin, development, and specialized Salesforce topics >

Here's everything — every probable question with complete, interview-ready answers.

PART A — ADMIN QUESTIONS

---

1. What is Salesforce?

Salesforce is a cloud-based CRM (Customer Relationship Management) platform. It helps businesses manage customer relationships, track sales, handle support cases, and automate business processes. It's both SaaS (ready-to-use apps like Sales Cloud, Service Cloud) and PaaS (you can build custom apps using Apex, Flows, and LWC). Everything runs on the cloud — no local installation needed.

---

2. What is CRM?

CRM stands for Customer Relationship Management. It is software that helps organizations store customer data, track every interaction (calls, emails, meetings), manage sales pipelines, and handle customer support — all in one central place.

---

3. Sales Cloud vs Service Cloud?

Sales Cloud — manages the sales process. Core objects: Lead, Account, Contact, Opportunity. Features: Web-to-Lead, Lead Assignment Rules, Sales Processes, Forecasting.

Service Cloud — manages customer support. Core objects: Case, Account, Contact. Features: Web-to-Case, Email-to-Case, Omni-Channel, Escalation Rules, Knowledge Base.

---

4. What is a Profile?

A Profile is a mandatory access template assigned to every user. Each user has exactly one profile. It controls:

- Object-level CRUD permissions (Create, Read, Edit, Delete)
- Field-Level Security (which fields are visible/editable)
- App and Tab visibility (Default On, Default Off, Tab Hidden)
- Login IP ranges and login hours

Examples: System Administrator, Standard User, Chatter Free.

---

5. What is a Permission Set?

A Permission Set grants additional permissions to a specific user on top of their profile. It only adds permissions — it never removes them. A user can have zero or many permission sets. A Permission Set Group bundles multiple permission sets for easier assignment.

Profile = baseline for everyone in that role. Permission Set = extras for specific individuals.

---

6. Profiles vs Permission Sets?

Aspect	Profile	Permission Set
Assignment	Mandatory — exactly 1 per user	Optional — 0 to many per user
Scope	Applies to all users with that profile	Applies to individual users
Purpose	Baseline access floor	Additional top-up permissions
Can remove access?	Yes — controls what's restricted	No — only adds, never restricts

7. What is a Record Type?

A Record Type allows you to show different page layouts and different picklist values to different users based on their profile. Use it when the same object needs different configurations for different teams.

Example: Account object with two Record Types — "City Corporation" and "NGO" — each showing different fields and picklist values.

---

8. What is OWD (Organization-Wide Defaults)?

OWD sets the most restrictive baseline access level for records a user doesn't own. It's the foundation of record-level security.

Options:

- **Private** — only owner and users above in Role Hierarchy can see
- **Public Read Only** — everyone can view, only owner can edit
- **Public Read/Write** — everyone can view and edit
- **Controlled by Parent** — child inherits parent's sharing (for Master-Detail)

Best practice: Set OWD as Private, then open access using Role Hierarchy and Sharing Rules.

---

9. What is Role Hierarchy?

Role Hierarchy gives users higher in the org chart automatic access to records owned by users below them. It opens access upward.

Key points:

- Works automatically for standard objects
- Can be disabled for custom objects using "Grant Access Using Hierarchy" checkbox
- **Role = which records you see. Profile = what you can do.**

---

10. What are Sharing Rules?

Sharing Rules extend access beyond OWD to specific user groups. They can only open up access — never restrict it.

Two types:

- **Owner-based** — shares based on who owns the record

- **Criteria-based** — shares based on field values (e.g., Severity = Critical)

You share with: Public Groups, Roles, or Roles and Subordinates.

---

11. Queues vs Public Groups?

Queues — used for load balancing. Can OWN records (Lead, Case, Custom Objects). Members pick up work from a shared pool. Visible as record owner in list views.

Public Groups — used for sharing rules, folder access, email alerts. CANNOT own records. Used to group users for collaboration.

---

12. What is Field-Level Security (FLS)?

FLS controls which fields a user can see or edit on an object. Configured at the Profile or Permission Set level. Options per field: Visible + Editable, Visible + Read Only, or Hidden entirely.

---

13. What is a Validation Rule?

A Validation Rule ensures data quality. It contains a formula — when the formula evaluates to TRUE, Salesforce shows an error message and prevents the record from being saved.

The formula must return TRUE when data is INVALID.

Examples:

- `Age__c < 18` → volunteer must be 18+
- `End_Date__c <= Start_Date__c` → end date must be after start
- `ISBLANK(Location_Area__c)` → location is required

---

14. What is a Flow? What are the types?

A Flow is Salesforce's declarative automation tool — automates business processes without code.

5 types:

- **Screen Flow** — interactive UI wizard for users
- **Record-Triggered Flow** — fires on record create/update/delete
- **Scheduled Flow** — runs at a specific time
- **Auto-Launched Flow** — background, no UI, called from Apex or subflow
- **Platform Event-Triggered Flow** — fires on platform event messages

Record-Triggered Flows run in System Context by default — they bypass sharing rules.

---

15. What is a Report Type? Report Formats?

Report Type — a template that defines which objects and fields are available when creating a report. You select it before building the report. Two kinds: Standard (pre-built by Salesforce) and Custom (admin creates for specific object relationships).

Report Formats — how data is displayed:

- **Tabular** — simple list, no grouping
- **Summary** — groups with subtotals (most common)

- **Matrix** — groups by rows AND columns
- **Joined** — multiple report blocks in one view

Report Type = which data. Report Format = how it looks.

---

16. What is a Dashboard? What is a Dynamic Dashboard?

Dashboard — visual representation of reports using charts. Max 20 components.

Running User — the user whose permissions determine what data appears.

Dynamic Dashboard — set running user to "The Dashboard Viewer" so each person sees only data they have access to. Each viewer sees their own data, not everyone's.

---

17. Data Import Wizard vs Data Loader?

Feature	Import Wizard	Data Loader
Interface	Browser-based	Desktop app
Record limit	50,000	5 million
Operations	Insert, Update, Upsert	Insert, Update, Upsert, Delete, Export
Best for	Quick small imports	Bulk migrations, deletions

18. What is an External ID?

A field storing a unique identifier from an external system. Enables upsert operations (update if exists, insert if new) and prevents duplicates during data migration. Supported on: Text, Number, Email fields.

---

19. Lookup vs Master-Detail Relationship?

Feature	Lookup	Master-Detail
Coupling	Loose — child can exist alone	Tight — parent is required
Cascade delete	No	Yes — delete master deletes all children
Roll-Up Summary	Not available	Available on master
Sharing/Owner	Independent owners	Child inherits master's sharing

Use Lookup when child has independent value. Use Master-Detail when child is meaningless without parent.

---

20. What is a Junction Object?

A custom object with two Master-Detail relationships that creates a Many-to-Many relationship between two objects.

Example: Volunteer_Participation__c with M-D to Volunteer__c and M-D to Green_Initiative__c. One volunteer joins many initiatives, one initiative has many volunteers.

---

21. What is a Roll-Up Summary Field?

A field on the MASTER object in a Master-Detail relationship that aggregates data from child records. Functions: COUNT, SUM, MIN, MAX. Only works on Master-Detail — for Lookup relationships, use Flow or Apex instead.

---

22. Custom Settings vs Custom Metadata?

Feature	Custom Settings	Custom Metadata
Data deploys?	No — structure only	Yes — records deploy too
Per-user config?	Yes (Hierarchy type)	No
In formulas?	Limited	Yes
Best for	Runtime config per environment	Config that must move between orgs

23. What is a Custom Label?

Stores reusable text values (messages, URLs) accessible in Apex, Flows, LWC, and Visualforce. Supports multi-language translation. Avoids hardcoding strings in code.

---

24. What is a Formula Field?

A read-only field that calculates its value automatically using a formula expression. Computed on the fly when the record is displayed — no data is stored. A Cross-Object Formula references fields from a parent object.

---

25. What is a Compact Layout?

Controls which fields appear in the Highlights Panel (top of record page), mobile views, and hover pop-ups. Shows the most important fields at a quick glance.

---

26. What are Assignment Rules?

Automatically route Lead or Case records to a specific User or Queue based on criteria. Only one rule active per object at a time. Rule entries evaluated in order — first match wins.

---

27. What is Field Dependency?

Links a Controlling Field to a Dependent Picklist. Values in the dependent picklist filter based on the controlling field's selection. Example: Issue_Category = Pollution → Sub_Type shows Air, Water, Soil.

PART B — APEX TRIGGERS

---

28. What is an Apex Trigger?

Code that executes automatically before or after DML events (insert, update, delete, undelete) on a Salesforce object.

Events: before insert, before update, before delete, after insert, after update, after delete, after undelete.

There is NO before undelete.

---

29. Trigger.new vs Trigger.old?

- **Trigger.new** — List of new/updated records. Available in: insert, update, undelete.
- **Trigger.old** — List of previous versions. Available in: update, delete.
- **Trigger.newMap** — Map<Id, SObject> of new records. Available in: after insert, before/after update.
- **Trigger.oldMap** — Map<Id, SObject> of old records. Available in: before/after update, before/after delete.

Trigger.new on before insert has no ID (not saved yet). On after insert, ID is populated.

---

30. Before Trigger vs After Trigger?

Before — fires before record is saved. Can modify Trigger.new directly without DML. Use for: field defaulting, validation. Record has no ID on insert.

After — fires after record is saved. Trigger.new is read-only — need explicit DML to update. Use for: creating related records, callouts. Record has ID.

---

31. What is Bulkification?

Writing trigger code that handles 200 records at once without hitting governor limits.

Rule: Never put SOQL or DML inside a for loop. Collect IDs into a Set, run one query outside the loop, use a Map for lookups.

Bad: `for(Case c : Trigger.new) { Contact con = [SELECT Id FROM Contact WHERE Id = :c.ContactId]; }`

Good: Collect all ContactIds → one SOQL → Map<Id, Contact> → loop and use map.

---

32. What is a Recursive Trigger? How to prevent?

A trigger fires, performs DML, which fires the same trigger again — infinite loop.

Prevention: Use a static Boolean variable in a helper class.

```
public class TriggerHelper { public static Boolean hasRun = false; }
// In trigger: if(!TriggerHelper.hasRun) { TriggerHelper.hasRun = true; /* logic */
}
```

Static variables persist within a transaction, reset between transactions. Each Data Loader batch of 200 is a new transaction — flag resets correctly.

---

33. One Trigger Per Object — why?

Salesforce doesn't guarantee execution order for multiple triggers on the same object. One trigger → one handler class keeps execution predictable, testable, and maintainable. The trigger is thin (just calls the handler). All logic is in the handler class organized by event.

---

34. What is the Order of Execution?

1. System validation
2. Before triggers
3. Validation Rules
4. Record saved to DB (not committed)
5. After triggers
6. Assignment Rules
7. Workflow Rules
8. Flows
9. Transaction committed

Key: Before triggers fire BEFORE validation rules. After triggers fire AFTER record save but before commit.

---

35. What is addError()?

Prevents a record from saving and shows a user-friendly error message. Marks only that specific record as failed — other records in the batch can still succeed (with allOrNone=false). Different from throwing an exception, which rolls back the entire transaction.

---

36. Governor Limits for Triggers?

Limit	Value
SOQL per transaction	100
DML per transaction	150
Records per DML	10,000
CPU time (sync)	10,000ms
Heap size (sync)	6MB
@future calls	50 per transaction

Why limits exist: Salesforce is multi-tenant — limits prevent one org from starving others.

---

37. Can a trigger make a callout?

No — triggers run inside a database transaction. A callout would block the transaction and cause timeouts. Solution: Use @future(callout=true) or Queueable with Database.AllowsCallouts to make the callout asynchronously after the transaction commits.

---

38. What is Trigger.operationType?

Returns a System.TriggerOperation enum (BEFORE_INSERT, AFTER_UPDATE, etc.). Cleaner than combining Trigger.isBefore && Trigger.isInsert. Use with a switch statement to route to handler methods.

---

39. How to bypass a trigger during data migration?

Use a Custom Setting with a bypass checkbox. The trigger checks the flag before running logic. Enable flag → run Data Loader → disable flag. Never delete or comment out a trigger for migration.

---

40. Can you modify Trigger.new in an after trigger?

No — Trigger.new is read-only in after context. You must create new sObject instances with just the Id and changed fields, add them to a list, and perform a separate update DML.

---

41. How to access parent fields in a trigger?

Trigger.new does NOT include parent data. You must run a SOQL query with relationship traversal: `SELECT Id, Contact.Name, Contact.Ward_Area__c FROM Case WHERE Id IN :caseIds`

---

42. Can a trigger be written on Custom Metadata?

No — Custom Metadata Types are metadata, not data. They don't support triggers. List Custom Settings can have triggers, but it's rare and not recommended.

---

43. Platform Events and Triggers?

Triggers can publish platform events using EventBus.publish(). You can also write a trigger on a platform event object (`trigger on Case_Alert__e (after insert)`). Used for real-time integrations and decoupled architecture.

PART C — ASYNCHRONOUS APEX

---

44. What is Asynchronous Apex?

Code that runs in a separate background thread, outside the user's transaction. It gets its own governor limits and doesn't block the UI. Needed for: external callouts, large data processing, scheduled jobs, and operations that exceed synchronous governor limits.

---

45. What are the 4 types of Async Apex?

@future — Simplest. Static, void, primitive parameters only. No job monitoring. No chaining. Max 50 per transaction. Use for simple callouts from triggers.

Queueable — Modern standard. Accepts sObjects. Returns Job ID. Supports chaining (5 levels). Use for anything @future can't handle.

Batch Apex — For millions of records. Three methods: start() collects records (50M max), execute() processes chunks of 200 (limits reset per chunk), finish() runs cleanup. Implement Database.Stateful to preserve variables across batches.

Scheduled Apex — Runs at specific times using cron expressions. Usually launches a Batch job. Cron format: Seconds Minutes Hours Day Month DayOfWeek.

---

46. @future — rules?

- Must be static
- Must return void

- Only primitive parameters (no sObjects)
- Cannot call @future from @future (hard limit)
- No Job ID returned
- Max 50 per transaction
- Add `(callout=true)` for HTTP callouts

---

47. Queueable vs @future?

Feature	@future	Queueable
Complex parameters	No — primitives only	Yes — sObjects, custom types
Job monitoring	No	Yes — returns Job ID
Job chaining	No	Yes — up to 5 levels
Callouts	Yes (callout=true)	Yes (Database.AllowsCallouts)

Queueable is the modern recommendation for most async use cases.

---

48. Batch Apex — start, execute, finish?

start() — runs once. Returns a QueryLocator (up to 50 million records) or Iterable. Collects all records to process.

execute() — runs N times. Processes one batch chunk (default 200, max 2000 records). Governor limits RESET for each execute call.

finish() — runs once after all batches complete. Used for sending emails, chaining another batch, or logging.

Database.Stateful — implement to preserve instance variables across batches (for counters).
Database.AllowsCallouts — implement to enable HTTP callouts in execute().

---

49. Scheduled Apex — cron syntax?

Format: Seconds Minutes Hours Day Month DayOfWeek

- `0 0 2 * * ?` → every day at 2 AM
- `0 30 8 ? * MON-FRI` → 8:30 AM weekdays
- `0 0 0 1 * ?` → midnight on 1st of every month

Common pattern: Scheduled Apex calls Database.executeBatch() inside execute(). Scheduled Apex is a launcher, not a processor.

---

50. Can @future call another @future?

No — hard platform limit. Throws: "Future method cannot be called from a future or batch method." Use Queueable chaining instead.

PART D — LWC (Lightning Web Components)

---

51. What is LWC?

Lightning Web Components is Salesforce's modern UI framework built on standard web technologies — HTML, JavaScript, CSS. It uses native browser APIs, making it faster than the older Aura framework. Salesforce recommends LWC for all new UI development.

---

52. What files make up an LWC?

- **component.html** — template markup (required)
- **component.js** — logic, state, lifecycle hooks (required)
- **component.css** — scoped styles, only apply to this component (optional)
- **component.js-meta.xml** — tells Salesforce where the component can be used (optional but almost always needed)

---

53. What are the 3 LWC Decorators?

@api — makes a property/method public. Enables parent → child communication. Read-only inside the child. Example: `@api recordId;`

@track — makes nested object/array properties reactive. Primitives are already reactive without @track. Only needed for detecting changes inside objects. Example: `@track caseData = {};`

@wire — reactive data service. Auto-fetches when input parameters change. Requires `@AuraEnabled(cacheable=true)` on Apex. Example: `@wire(getCases, { ward: '$selectedWard' }) cases;`

The `$` prefix makes the parameter reactive — when the property changes, @wire re-fetches automatically.

---

54. What is @api in detail?

@api makes a property or method accessible to the parent component.

For properties: parent sets value in template `<c-child case-id={myId}>`. Child declares `@api caseId;`.

For methods: parent calls `this.template.querySelector('c-child').refreshData()`. Child declares `@api refreshData() {}`.

@api properties are read-only inside the child. To modify, copy to a local property first.

---

55. Lifecycle Hooks — in order?

1. **constructor()** — component created. Call super() first. DOM not ready.
2. **connectedCallback()** — inserted into DOM. Fetch data here. Most commonly used hook.
3. **renderedCallback()** — after every render/re-render. Use guards to prevent infinite loops.
4. **disconnectedCallback()** — removed from DOM. Cleanup event listeners.
5. **errorCallback(error, stack)** — catches errors from child components.

---

56. @wire vs Imperative Apex?

@wire — reactive. Auto-fetches when inputs change. Apex must have `cacheable=true`. Results may be cached. Cannot be used with DML methods.

Imperative — on-demand. Call explicitly on button click. Works with DML methods. Clean .then()/.catch() error handling. You control exactly when it fires.

Use @wire for read-only data that should refresh automatically. Use imperative for DML, user actions, and conditional calls.

---

57. What is Lightning Data Service (LDS)?

LDS lets you do CRUD on Salesforce records directly from LWC without writing Apex. Methods: createRecord(), getRecord(), updateRecord(), deleteRecord() from `lightning/uiRecordApi`.

3 advantages over Apex:

1. **Automatic FLS and sharing** — no extra security code needed
2. **Client-side caching** — shared cache across components on the same page
3. **Zero maintenance** — no Apex class to write, test, or deploy

Use LDS for simple single-record CRUD. Use Apex for complex queries, multi-object ops, or business logic.

---

58. Component Communication — all 3 patterns?

Parent → Child — use @api. Parent sets property in template or calls @api method via querySelector.

Child → Parent — child dispatches CustomEvent using `this.dispatchEvent(new CustomEvent('caseselected', { detail: data }))`. Parent listens with `oncaseselected={handler}` in template. Event names must be lowercase, no hyphens.

Unrelated components — Lightning Message Service (LMS). Publish/subscribe model using a Message Channel. Works across LWC, Aura, and Visualforce.

---

59. What is Shadow DOM?

Every LWC has its own Shadow DOM — an encapsulated piece of the page that isolates its CSS and DOM from other components. Styles in one component don't leak into others. Use `this.template.querySelector()` to access your own DOM (not `document.querySelector()`). Use CSS custom properties (variables) to allow parent styling of child components.

---

60. What is refreshApex()?

Forces a wired property to re-fetch data from the server after a DML operation, bypassing the cache. Without it, the wired property shows stale cached data after you create/update/delete records.

```
await updateCase({ caseId: this.selectedId });
return refreshApex(this.wiredCases);
```

---

61. What is NavigationMixin?

Used to navigate to record pages, list views, or external URLs from LWC. You extend your class with `NavigationMixin(LightningElement)` and call `this[NavigationMixin.Navigate]()` with a page reference object specifying the type, recordId, and action.

62. What is a Toast message?

A notification popup shown using `ShowToastEvent` from `lightning/platformShowToastEvent`. Variants: success, error, warning, info. Note: Toast does NOT work in Experience Cloud sites.

---

63. for:each vs iterator?

for:each — simple loop. Gives each item. Must provide a unique `key` attribute. No way to detect first/last item.

iterator — gives extra properties: value, index, first, last. Use when you need to style first or last item differently.

---

64. What is the js-meta.xml file?

Metadata config file that controls where the component can be used. Key properties: `isExposed=true` (makes it visible in App Builder), `targets` (lightning__RecordPage, lightning__AppPage, lightning__HomePage, lightning__FlowScreen, lightningCommunity__Page).

---

65. Forms in LWC — 3 approaches?

1. **lightning-record-form** — simplest. Auto-generates form. Least control.
2. **lightning-record-edit-form + lightning-input-field** — you choose which fields and order. Still auto-handles save/error. Best balance.
3. **Custom form with lightning-input** — full manual control. Call Apex imperatively. Most code.

---

66. What does cacheable=true mean?

Marks an Apex method as read-only and safe for client-side caching. Required for @wire. Results may be served from browser cache. Cannot be used on methods that perform DML.

PART E — APEX TESTING

---

67. What is @isTest?

Marks a class/method as test code. Not deployed to production. Not counted toward coverage. Runs in isolated data context (SeeAllData=false by default). Has access to all classes via @TestVisible.

---

68. What is the 75% coverage rule?

Production: minimum 75% org-wide coverage required to deploy. Every trigger needs at least 1%. Sandbox: no minimum required.

The 75% applies to the ENTIRE ORG — not just the classes being deployed. If your deployment drops the org total below 75%, it's blocked.

---

69. System.assert() vs assertEquals()?

System.assert(condition) — checks boolean is true. Failure message: "Assertion Failed" — no detail.

System.assertEquals(expected, actual) — checks exact equality. Failure message: "Expected: In Progress, Actual: Open" — tells you exactly what's wrong.

Always prefer assertEquals — detailed failure messages save debugging time.

---

70. Test.startTest() and Test.stopTest()?

startTest() — resets governor limits. Code inside gets a fresh set of limits separate from data setup.

stopTest() — forces all async operations (@future, Queueable, Batch) to complete synchronously before assertions run.

Only ONE startTest/stopTest pair per test method.

---

71. What is SeeAllData?

Default is `SeeAllData=false` — tests can't see real org data, must create their own. `SeeAllData=true` allows access to live data — avoid this because tests become fragile and slow.

---

72. What is @testSetup?

A method that runs ONCE before all test methods in the class. Creates shared test data. More efficient than creating data in every test method. Each test method gets a fresh copy — changes in one test don't affect others.

---

73. What is a Test Data Factory?

An @isTest utility class with static methods that create standardized test records. Shared across all test classes. When a validation rule changes, fix one factory method instead of 20 test classes. Centralized, DRY, consistent.

---

74. How to test HTTP callouts?

Salesforce blocks real HTTP calls in tests. Use HttpCalloutMock interface — create a mock class that returns a fake response. Register with `Test.setMock(HttpCalloutMock.class, new MockClass())` BEFORE Test.startTest().

---

75. What is System.runAs()?

Executes test code as a specific user, enforcing their sharing rules and record visibility. Does NOT enforce FLS or object-level CRUD permissions — only record-level sharing.

---

76. What is @TestVisible?

Lets test classes access private methods and properties without changing them to public. Keeps production code properly encapsulated while allowing thorough unit testing.

---

77. Why test 200 records?

Salesforce sends up to 200 records per trigger invocation. Testing with 1 record doesn't expose SOQL-in-loop bugs. Testing with 200 proves your code is bulkified and won't hit governor limits in production.

---

78. What is the Mixed DML error?

Occurs when DML on Setup Objects (User, Profile) and non-Setup Objects (Account, Case) happens in the same transaction. Fix: use System.runAs() to separate them into different execution contexts.

---

79. How to test SOSL?

SOSL returns empty results in tests by default. Use `Test.setFixedSearchResults(new Id[]{recordId})` to pre-define what SOSL should return. Call it before executing the SOSL query.

---

80. How to test Scheduled Apex?

Call `System.schedule()` inside startTest/stopTest. Capture the returned Job ID. Query CronTrigger to verify State = 'WAITING' and CronExpression matches.

PART F — PROJECT-SPECIFIC QUESTIONS

---

81. Explain your project.

EcoConnect is a Community Sustainability Platform built on Salesforce Experience Cloud. Citizens report environmental issues as Cases. Volunteers join Green Initiatives like clean-up campaigns. City Officers manage and resolve everything. Uses standard objects (Account, Contact, Case) extended with custom fields, plus 4 custom objects (Volunteer__c, Green_Initiative__c, Volunteer_Participation__c, Feedback__c). Automation handles case routing, volunteer flagging, and status tracking. Reports and dashboards give officers data-driven insights.

---

82. Why Contact for Citizens, not a Custom Object?

Contact is a standard object optimized by Salesforce — supports Experience Cloud login, has built-in relationship to Account, works natively with Cases, and supports email, activities, and reports. A custom object would mean rebuilding all of that from scratch.

---

83. Why Master-Detail for Contact → Volunteer__c?

A Volunteer record is meaningless without a Citizen. If a citizen is deleted, volunteer data should go too (cascade delete). Master-Detail also enables Roll-Up Summaries like Total_Campaigns__c on Contact.

---

84. Why Lookup for Account → Green_Initiative__c?

Initiatives have historical and community value. If an NGO Account is deleted, the initiative record should survive with its participation data and completion history. Lookup preserves independence.

---

85. Why Lookup for Feedback__c?

Feedback is permanent evidence. If a Case is closed/deleted or a Volunteer becomes inactive, the feedback must survive for audit and reporting. Lookups ensure feedback persists even when parent records are removed.

---

86. Why Flow for Is_Volunteer__c and not a Formula?

A formula on Contact cannot look downward to check if a child Volunteer__c record exists — formulas reference the same object or parents, not children. A Record-Triggered Flow on Volunteer__c (after create) is the cleanest approach — it sets Is_Volunteer__c = TRUE on the parent Contact.

---

87. What automations did you build?

- **Record-Triggered Flow** on Volunteer__c (after insert) → sets Is_Volunteer__c = TRUE on parent Contact
- **Validation Rules** → Age must be 18+, End Date after Start Date, Location required on Cases
- **Assignment Rules** → Cases auto-routed to officer queues by Severity (Critical → Senior Queue, High → Zone Queue)
- **Flow/Email Alert** → Notify officer when a critical case is created

---

88. What security model did you implement?

- OWD set to Private for Cases
- Role Hierarchy: City Officers above Citizens/Volunteers — officers see citizen data
- Sharing Rules: Critical-severity cases shared with Senior Officer public group
- FLS: GPS coordinates hidden from citizens, visible to officers only
- Record Types: Account has City_Corporation and NGO record types with different layouts

---

That's 88 questions covering every probable topic. Print this out or keep it open on your phone — read through it 2-3 times before the interview. Focus especially on the ones you scored low on in the mock: **Report Types, Async Apex details, and LDS advantages**. Good luck, Vedant.

Also add a PART-G for SOQL,SOSL

Orchestrated comprehensive SOQL and SOSL interview content compilation >

PART G — SOQL & SOSL

---

89. What is SOQL?

SOQL stands for Salesforce Object Query Language. It is used to query records from a single object or related objects in Salesforce. It is similar to SQL's SELECT statement but is designed specifically for Salesforce's data model.

Key difference from SQL: SOQL cannot query all objects at once — you must specify one primary object. No `SELECT *` — you must name every field explicitly.

```
SELECT Id, Subject, Status, Contact.Name
FROM Case
WHERE Status = 'Open'
ORDER BY CreatedDate DESC
LIMIT 100
```

---

90. What is SOSL?

SOSL stands for Salesforce Object Search Language. It is used to search for text across MULTIPLE objects simultaneously. It searches indexed fields and returns results grouped by object.

```
FIND {Pollution} IN ALL FIELDS
RETURNING Case(Id, Subject), Contact(Id, Name)
```

Key difference from SOQL: SOQL queries one object at a time with exact filters. SOSL searches across many objects at once using text-based search (like a search engine).

---

91. SOQL vs SOSL — when to use which?

Aspect	SOQL	SOSL
Purpose	Query specific records from one object (with related objects)	Search text across multiple objects simultaneously
Knows which object?	Yes — you query a specific object	Not necessarily — you're searching broadly
Knows which field?	Yes — you filter on exact fields	No — searches all indexed fields
Returns	List of records from one object	List of Lists — grouped by object
Use when	You know which object and which field contains the data	You have a search term and want to find it anywhere
Example	"Get all open Cases in Zone A"	"Find any record containing the word 'Pollution'"
Governor limit	100 queries per transaction	20 searches per transaction

Rule of thumb: If you know the object and field → SOQL. If you have a keyword and want to search everywhere → SOSL.

---

92. What are the SOQL Governor Limits?

Limit	Synchronous	Asynchronous
Total SOQL queries per transaction	100	200
Total records returned by SOQL	50,000	50,000

Limit	Synchronous	Asynchronous
Total SOSL searches per transaction	20	20
Records returned by SOSL	2,000	2,000
QueryLocator max (Batch Apex)	50 million	50 million

93. What are the different SOQL clauses?

- **SELECT** — which fields to retrieve (mandatory)
- **FROM** — which object to query (mandatory)
- **WHERE** — filter conditions
- **ORDER BY** — sort results (ASC or DESC)
- **LIMIT** — max number of records to return
- **OFFSET** — skip a number of records (for pagination)
- **GROUP BY** — group results for aggregate queries
- **HAVING** — filter on aggregate results (used with GROUP BY)
- **WITH SECURITY_ENFORCED** — enforces FLS, throws error if user lacks access

```
SELECT Status, COUNT(Id) total
FROM Case
WHERE CreatedDate = THIS_YEAR
GROUP BY Status
HAVING COUNT(Id) > 10
ORDER BY COUNT(Id) DESC
LIMIT 5
```

---

94. What are Relationship Queries in SOQL?

You can query related objects in SOQL using two types of relationship queries:

Child-to-Parent (dot notation) — traverse upward to the parent:

```
SELECT Id, Subject, Contact.Name, Contact.Ward_Area__c
FROM Case
WHERE Contact.Ward_Area__c = 'Zone A'
```

For custom relationships, use `__r`: `Volunteer__r.Name`

Parent-to-Child (subquery) — traverse downward to children:

```
SELECT Id, Name,
    (SELECT Id, Subject, Status FROM Cases)
FROM Account
WHERE Name = 'Chennai City Corp'
```

For custom relationships, use the child relationship name with `__r`: `(SELECT Id FROM Volunteer_Participations__r)`

Key rule: Child-to-parent uses dot notation. Parent-to-child uses a subquery in parentheses. You can traverse up to 5 levels in either direction.

---

95. What is a Subquery?

A subquery is a nested SELECT statement inside a parent SOQL query that retrieves related child records. It always appears in parentheses inside the parent's SELECT clause.

```
SELECT Id, Name,
    (SELECT Id, FirstName, LastName FROM Contacts),
    (SELECT Id, CaseNumber, Status FROM Cases)
FROM Account
WHERE Id = '001XXXXXXXXXXX'
```

This returns each Account with its related Contacts and Cases in a single query — more efficient than running separate queries.

---

96. What are Aggregate Functions in SOQL?

SOQL supports these aggregate functions with GROUP BY:

- **COUNT()** — number of records
- **COUNT(fieldName)** — number of non-null values in that field
- **SUM(fieldName)** — total of a numeric field
- **AVG(fieldName)** — average of a numeric field
- **MIN(fieldName)** — smallest value
- **MAX(fieldName)** — largest value
- **COUNT_DISTINCT(fieldName)** — number of unique non-null values

```
SELECT Issue_Category__c, COUNT(Id) totalCases, AVG(Resolution_Days__c) avgDays
FROM Case
GROUP BY Issue_Category__c
```

Results are returned as `List<AggregateResult>`. Access values using `.get('alias')` or `.get('expr0')`.

apex

```apex
List<AggregateResult> results = [SELECT Status, COUNT(Id) cnt FROM Case GROUP BY Statu
for (AggregateResult ar : results) {
    String status = (String) ar.get('Status');
    Integer count = (Integer) ar.get('cnt');
}
```

---

97. What are Date Literals in SOQL?

Date literals are pre-built keywords that represent date ranges — no hardcoding needed:

Literal	Meaning
TODAY	Today's date
YESTERDAY	Yesterday
THIS_WEEK	Current week (Sun–Sat)
LAST_WEEK	Previous week
THIS_MONTH	Current calendar month

Literal	Meaning
LAST_MONTH	Previous month
THIS_YEAR	Current calendar year
LAST_N_DAYS:30	Last 30 days from today
NEXT_N_DAYS:7	Next 7 days from today
LAST_N_MONTHS:3	Last 3 calendar months

```
SELECT Id, Subject FROM Case
WHERE CreatedDate = THIS_MONTH
AND Severity_Level__c = 'Critical'
```

---

98. What is Dynamic SOQL?

Dynamic SOQL is building a SOQL query as a String at runtime and executing it using `Database.query()`. Useful when filters, fields, or objects are determined at runtime.

```apex
String objectName = 'Case';
String query = 'SELECT Id, Subject, Status FROM ' + objectName +
               ' WHERE Status = :statusFilter LIMIT 100';
List<SObject> results = Database.query(query);
```

Risks: Vulnerable to SOQL injection if user input is concatenated directly. Always use bind variables (`:variableName`) instead of string concatenation for filter values.

Static SOQL (inline, preferred):

```apex
List<Case> cases = [SELECT Id FROM Case WHERE Status = :myVar];
```

Dynamic SOQL (use only when necessary):

```apex
List<SObject> results = Database.query(queryString);
```

Static is safer and faster. Use dynamic only when the object, fields, or structure must change at runtime.

---

99. What is the difference between WHERE and HAVING?

WHERE — filters individual records BEFORE grouping.

HAVING — filters aggregated results AFTER grouping (used with GROUP BY).

```
SELECT Issue_Category__c, COUNT(Id) total
FROM Case
WHERE Status != 'Closed'          -- filters before grouping
GROUP BY Issue_Category__c
HAVING COUNT(Id) > 5              -- filters after grouping
```

Think of it as: WHERE filters rows → GROUP BY groups them → HAVING filters groups.

---

100. What are SOQL Bind Variables?

Bind variables let you use Apex variables directly in SOQL queries using the colon (`:`) prefix. This is safer than string concatenation and prevents SOQL injection.

```apex
String ward = 'Zone A';
String status = 'Open';
List<Case> cases = [SELECT Id, Subject
                    FROM Case
                    WHERE Location_Area__c = :ward
                    AND Status = :status];
```

You can bind: single variables, Lists, Sets, and Maps (keySets).

```apex
Set<Id> contactIds = new Set<Id>{'003XX...', '003YY...'};
List<Contact> contacts = [SELECT Id, Name FROM Contact WHERE Id IN :contactIds];
```

---

101. What is the SOQL FOR UPDATE clause?

`FOR UPDATE` locks the queried records for the duration of the transaction, preventing other users/processes from updating them simultaneously. Used to prevent race conditions.

```apex
List<Case> cases = [SELECT Id, Status FROM Case WHERE Id = :caseId FOR UPDATE];
// These records are now locked — other transactions must wait
cases[0].Status = 'In Progress';
update cases;
// Lock released when transaction commits
```

Use carefully — locked records block other users. Only use when you need to guarantee no concurrent edits during your transaction.

---

102. What is TYPEOF in SOQL? (Polymorphic Relationships)

TYPEOF handles polymorphic lookup fields — fields that can point to different object types (like WhoId on Task can be a Contact or Lead).

```
SELECT Id, Subject,
    TYPEOF Who
        WHEN Contact THEN Name, Email, Phone
        WHEN Lead THEN Name, Company
    END
FROM Task
```

Returns different fields depending on which object type the polymorphic field points to.

---

103. What are the SOSL search groups?

SOSL's `IN` clause specifies which fields to search:

- **ALL FIELDS** — searches all searchable fields (name, email, phone, custom fields)
- **NAME FIELDS** — searches only name fields
- **EMAIL FIELDS** — searches only email fields
- **PHONE FIELDS** — searches only phone fields

```
FIND {Vedant} IN NAME FIELDS
RETURNING Contact(Id, Name, Email), Lead(Id, Name, Company)
```

---

104. SOSL — RETURNING clause?

The RETURNING clause specifies which objects and fields to include in the results, plus optional filters:

```
FIND {Pollution Zone A} IN ALL FIELDS
RETURNING
    Case(Id, Subject, Status WHERE Status = 'Open' ORDER BY CreatedDate DESC LIMIT
10),
    Contact(Id, Name WHERE Ward_Area__c = 'Zone A'),
    Green_Initiative__c(Id, Name)
```

You can add WHERE, ORDER BY, and LIMIT per object inside RETURNING — this filters the search results further.

---

105. How do you test SOSL in Apex tests?

SOSL returns empty results in test context by default. Use `Test.setFixedSearchResults()` to pre-define which record IDs SOSL should return:

apex

```apex
@isTest
static void testSearch() {
    Contact c = new Contact(LastName='TestCitizen');
    insert c;

    Test.setFixedSearchResults(new Id[]{ c.Id });

    List<List<SObject>> results =
        [FIND 'TestCitizen' IN ALL FIELDS RETURNING Contact(Id, LastName)];

    System.assertEquals(1, results[0].size());
}
```

Call `Test.setFixedSearchResults()` before executing the SOSL. Only returns records that actually exist in test data.

---

106. What is SOQL Injection? How to prevent it?

SOQL injection happens when user input is concatenated directly into a dynamic SOQL string, allowing malicious input to modify the query logic.

Vulnerable code:

```apex
String userInput = 'Open\' OR Status !=\'';  // malicious input
String query = 'SELECT Id FROM Case WHERE Status = \'' + userInput + '\'';
// Results in: WHERE Status = 'Open' OR Status !='' → returns ALL records
```

Prevention methods:

1. **Bind variables** (best) — `WHERE Status = :userInput` — Salesforce escapes automatically
2. **String.escapeSingleQuotes()** — escapes single quotes in user input
3. **Static SOQL** (inline) — always preferred over dynamic when possible

```apex
// SAFE — bind variable
List<Case> cases = [SELECT Id FROM Case WHERE Status = :userInput];

// SAFE — escapeSingleQuotes for dynamic SOQL
String safe = String.escapeSingleQuotes(userInput);
String query = 'SELECT Id FROM Case WHERE Status = \'' + safe + '\'';
```

---

107. What is WITH SECURITY_ENFORCED?

A SOQL clause that enforces Field-Level Security. If the running user doesn't have access to any field in the query, Salesforce throws an `System.QueryException` instead of silently returning the data.

```apex
List<Case> cases = [SELECT Id, Subject, GPS_Coordinates__c
                    FROM Case
                    WITH SECURITY_ENFORCED];
// If the user can't see GPS_Coordinates__c → exception thrown
```

Use this in Apex controllers for LWC to ensure FLS is respected. Without it, Apex runs in system mode and returns all fields regardless of the user's profile.

---

108. What is the difference between COUNT() and COUNT(fieldName)?

COUNT() — returns the total number of records matching the query. Cannot be used with other fields in SELECT (unless using GROUP BY).

```apex
Integer total = [SELECT COUNT() FROM Case WHERE Status = 'Open'];
```

COUNT(fieldName) — returns the number of records where that specific field is NOT null. Can be used with GROUP BY and other aggregate functions.

```apex
List<AggregateResult> results = [
    SELECT Issue_Category__c, COUNT(Resolution_Date__c) resolved
    FROM Case
    GROUP BY Issue_Category__c
```

```
];
// Counts only cases where Resolution_Date__c is not null
```

---

109. SOQL Best Practices for Interviews?

1. **Never put SOQL inside a loop** — collect IDs in a Set, query once outside the loop, use a Map
2. **Always specify fields** — no `SELECT *` in SOQL; query only what you need
3. **Use bind variables** — prevents SOQL injection, cleaner code
4. **Use LIMIT** — protect against accidentally returning 50,000 records
5. **Use WHERE filters** — narrow down results as much as possible
6. **Use WITH SECURITY_ENFORCED** — enforce FLS in Apex controllers
7. **Use selective filters** — filter on indexed fields (Id, Name, RecordType, Foreign Keys, External IDs) for better performance
8. **Use relationship queries** — one parent-to-child subquery is better than two separate queries

---

110. Quick Comparison Summary

Feature	SOQL	SOSL
Query type	Structured query on one object	Text search across multiple objects
Syntax starts with	SELECT	FIND
Returns	List<SObject>	List<List<SObject>>
Governor limit	100 per transaction	20 per transaction
Max records returned	50,000	2,000
Relationships	Yes — parent-child subqueries, dot notation	No — only flat results per object
Aggregate functions	Yes — COUNT, SUM, AVG, MIN, MAX	No
Dynamic version	Database.query(string)	Search.query(string)
Test support	Works normally with test data	Needs Test.setFixedSearchResults()
Use when	You know the object and field	You have a keyword, search broadly

That's 22 SOQL/SOSL questions (Q89–Q110) covering everything from basics to injection prevention. Your total is now **110 questions** across all 7 parts. The SOQL topics most likely to come up in your interview are: **SOQL vs SOSL difference, relationship queries, aggregate functions, bind variables, and governor limits.** Make sure you can explain those five cold.

PART-H for Governor Limits

Orchestrated comprehensive Salesforce Governor Limits coverage for interviews >

PART H — GOVERNOR LIMITS

111. What are Governor Limits? Why do they exist?

Governor Limits are runtime limits enforced by Salesforce on every Apex transaction. They restrict the amount of resources (database queries, DML operations, CPU time, memory) that any single transaction can consume.

Why they exist: Salesforce is a multi-tenant platform — thousands of organizations share the same physical servers, database, and infrastructure. Without limits, one org's runaway code could consume all resources and crash the platform for everyone else. Governor limits ensure fair usage and platform stability.

Key concept: Limits are enforced PER TRANSACTION, not per user or per org. A transaction is everything that happens from the time a user clicks Save to the time the database commits.

112. What are the key Governor Limits every developer must know?

Limit	Synchronous	Asynchronous
SOQL queries per transaction	100	200
Records retrieved by SOQL	50,000	50,000
SOSL searches per transaction	20	20
Records retrieved by SOSL	2,000	2,000
DML statements per transaction	150	150
Records processed by DML	10,000	10,000
Callouts (HTTP) per transaction	100	100
Total callout timeout	120 seconds	120 seconds
CPU time	10,000 ms	60,000 ms
Heap size	6 MB	12 MB
@future calls per transaction	50	0 (can't call from async)
Queueable jobs enqueued	50	1
SendEmail calls	10	10
QueryLocator max records (Batch)	—	50 million

Memorize these 6 at minimum: **100 SOQL, 150 DML, 50,000 rows, 10,000 DML records, 10s CPU, 6MB heap.**

113. What is a Transaction in Salesforce?

A transaction is a single unit of work — everything that happens from the initial trigger event (a user saves a record, an API call, a batch execute) until the final database commit.

Within one transaction, Salesforce executes:

- All before triggers
- Validation rules
- All after triggers
- Workflow rules
- Flows
- Assignment rules

All of these share the SAME set of governor limits. If a before trigger uses 80 SOQL queries, the after trigger, flows, and workflows only have 20 left.

When does a new transaction start?

- Each API call is its own transaction
- Each Batch execute() call is its own transaction
- Each @future or Queueable execution is its own transaction
- Data Loader — each batch of 200 records is its own transaction

---

114. What happens when a Governor Limit is exceeded?

Salesforce immediately throws a runtime exception, and the **entire transaction is rolled back**. No partial saves — all DML operations within that transaction are undone. The user sees an error message.

Common exceptions:

- `System.LimitException: Too many SOQL queries: 101` — exceeded 100 SOQL
- `System.LimitException: Too many DML statements: 151` — exceeded 150 DML
- `System.LimitException: Apex CPU time limit exceeded` — exceeded 10 seconds
- `System.LimitException: Apex heap size too large` — exceeded 6MB

LimitException is special — it CANNOT be caught by try-catch. When a LimitException fires, the transaction is dead. This is intentional — Salesforce forces you to fix the root cause rather than swallowing the error.

---

115. How do you check current limit consumption in Apex?

Use the `Limits` class — it has methods for every governor limit:

apex

```
System.debug('SOQL used: ' + Limits.getQueries() + ' / ' + Limits.getLimitQueries());
System.debug('DML used: ' + Limits.getDmlStatements() + ' / ' + Limits.getLimitDmlStat
System.debug('Rows used: ' + Limits.getDmlRows() + ' / ' + Limits.getLimitDmlRows());
System.debug('CPU time: ' + Limits.getCpuTime() + ' / ' + Limits.getLimitCpuTime());
System.debug('Heap used: ' + Limits.getHeapSize() + ' / ' + Limits.getLimitHeapSize())
```

Method	What It Returns
Limits.getQueries()	Number of SOQL queries used so far
Limits.getLimitQueries()	Max SOQL queries allowed (100 or 200)
Limits.getDmlStatements()	Number of DML statements used so far

Method	What It Returns
Limits.getDmlRows()	Number of records processed by DML so far
Limits.getCpuTime()	CPU time consumed in milliseconds
Limits.getHeapSize()	Heap memory used in bytes
Limits.getCallouts()	Number of HTTP callouts made so far

Use these in debug logs during development to monitor limit consumption before you hit production.

---

116. SOQL-inside-a-loop — the #1 Governor Limit violation. How to fix?

The problem: Every SOQL query inside a loop counts against the 100-query limit. With 200 records in a trigger, that's 200 queries — instant failure.

Bad code (hits limit at 101 records):

apex

```apex
for (Case c : Trigger.new) {
    Contact con = [SELECT Id, Name FROM Contact WHERE Id = :c.ContactId];
    // 200 records = 200 SOQL queries = CRASH
}
```

Fixed code (1 query regardless of record count):

apex

```apex
// Step 1: Collect all IDs
Set<Id> contactIds = new Set<Id>();
for (Case c : Trigger.new) {
    if (c.ContactId != null) contactIds.add(c.ContactId);
}

// Step 2: One query outside the loop
Map<Id, Contact> contactMap = new Map<Id, Contact>(
    [SELECT Id, Name FROM Contact WHERE Id IN :contactIds]
);

// Step 3: Use the Map inside the loop
for (Case c : Trigger.new) {
    Contact con = contactMap.get(c.ContactId);
    // No SOQL here — uses the Map
}
```

This pattern applies equally to DML. Never put insert, update, or delete inside a loop. Collect records into a List, then do one DML statement outside the loop.

---

117. DML-inside-a-loop — how to fix?

Bad code (hits 150 DML limit):

apex

```
for (Case c : Trigger.new) {
    Task t = new Task(Subject='Follow up', WhatId=c.Id);
    insert t;  // Each iteration = 1 DML statement → 200 DML for 200 records
}
```

Fixed code (1 DML regardless of record count):

```apex
List<Task> tasks = new List<Task>();
for (Case c : Trigger.new) {
    tasks.add(new Task(Subject='Follow up', WhatId=c.Id));
}
if (!tasks.isEmpty()) {
    insert tasks;  // One DML for all 200 tasks
}
```

Rule: Collect everything into a List inside the loop. Do one DML after the loop ends.

---

118. What is the 50,000 row limit? How to handle large data?

A single transaction can retrieve a maximum of 50,000 records total across ALL SOQL queries combined. If your queries return more than 50,000 rows collectively, you hit `System.LimitException: Too many query rows`.

Solutions for large data:

- **Use selective filters** in WHERE clause to narrow results — filter on indexed fields (Id, Name, RecordTypeId, Foreign Keys, External IDs)
- **Use LIMIT clause** — `SELECT Id FROM Case LIMIT 1000`
- **Use Batch Apex** — QueryLocator in Batch bypasses the 50,000 limit and can handle up to 50 million records
- **Use FOR loops with SOQL** — for processing large result sets without loading all into memory:

```apex
for (List<Case> batch : [SELECT Id, Status FROM Case WHERE Status = 'Open']) {
    // Processes in chunks of 200 automatically
    // Doesn't load all records into memory at once
    processCases(batch);
}
```

This SOQL-for-loop pattern retrieves records in batches of 200 — protecting against heap size limits.

---

119. What is the CPU Time Limit? What causes it?

Synchronous Apex has a 10,000 ms (10 second) CPU time limit. Asynchronous gets 60,000 ms (60 seconds).

What counts as CPU time:

- All Apex execution (loops, calculations, string operations)
- Trigger logic
- Flow execution

- Validation rule evaluation
- Formula field calculations

What does NOT count:

- Database query time (SOQL wait time)
- HTTP callout wait time
- Time waiting for locks

Common causes of CPU timeout:

- Nested loops (loop inside a loop inside a loop) — $O(n^3)$ complexity
- Complex string manipulation on large data sets
- Recursive triggers causing the same logic to run multiple times
- Too many workflows + flows + triggers on the same object compounding

How to fix:

- Reduce loop nesting — flatten to O(n) using Maps
- Move heavy processing to async (Batch or Queueable)
- Use Collections (Map, Set) instead of nested loops for lookups
- Minimize the logic that runs per record

---

120. What is the Heap Size Limit? How to manage memory?

Heap is the total memory used by all variables in your transaction. Synchronous limit is 6 MB, async is 12 MB.

What consumes heap:

- Every variable, object, list, map, and string in memory
- Large SOQL result sets loaded into Lists
- String concatenation in loops (each concatenation creates a new String)

How to manage:

- Query only the fields you need — `SELECT Id, Name` not `SELECT Id, Name, Description, LongTextArea__c`
- Use SOQL for-loops to process in chunks instead of loading everything into a List
- Nullify large variables after use — `largeList = null;`
- Avoid building large strings in loops — use a List and join at the end:

```apex
// BAD — each += creates a new String, old ones stay in heap
String result = '';
for (Case c : cases) {
    result += c.Subject + '\n';  // heap grows with every iteration
}

// GOOD — List is more memory-efficient
List<String> parts = new List<String>();
for (Case c : cases) {
    parts.add(c.Subject);
```

```
}
String result = String.join(parts, '\n');  // one String created at the end
```

---

121. What are the Callout Limits?

Limit	Value
Maximum callouts per transaction	100
Maximum timeout per single callout	120 seconds (2 minutes)
Maximum total callout time per transaction	120 seconds
Maximum request/response size	6 MB per callout
Callouts from triggers	NOT allowed directly — use @future or Queueable
Callouts from Batch execute()	Allowed with Database.AllowsCallouts

Why can't triggers make callouts? Triggers run inside a database transaction. A callout would block the transaction, potentially causing timeouts and data inconsistency. Solution: @future(callout=true) or Queueable with Database.AllowsCallouts.

---

122. What is the difference between Synchronous and Asynchronous Limits?

Limit	Sync	Async	Why async gets more
SOQL queries	100	200	No user is waiting — can do more work
CPU time	10,000 ms	60,000 ms	Background execution has more room
Heap size	6 MB	12 MB	More memory for large data processing
Queueable jobs	50	1	Prevents async chain explosion

Async Apex (future, Queueable, Batch, Scheduled) runs in its own transaction with higher limits because no user is waiting for an immediate response. This is why you offload heavy processing to async.

Batch Apex is special: Each execute() call gets its own fresh set of limits. So even if you're processing 1 million records, each chunk of 200 gets a full 100 SOQL, 150 DML, etc.

---

123. How does Batch Apex handle Governor Limits differently?

Batch Apex is specifically designed to work around governor limits:

- **start()** — uses QueryLocator which can return up to 50 million records (bypasses the 50,000 row limit)
- **execute()** — each batch chunk (default 200 records) gets a **completely fresh set of governor limits**. 100 SOQL, 150 DML, 10,000 records — all reset per batch.
- **finish()** — gets its own fresh limits too

This means: if you have 1 million records, and batch size is 200, execute() runs 5,000 times. Each of those 5,000 runs gets a full set of limits — it's like running 5,000 separate

transactions.

This is why Batch Apex is the solution for large data volumes that would exceed synchronous limits.

---

124. What are the Email Sending Limits?

Limit	Value
Single email messages per transaction	10
Single email messages per day (org)	5,000 (varies by org edition)
Mass email per day	5,000 external recipients
Email attachments max size	25 MB total per email

If you need to send more than 10 emails per transaction, use Batch Apex — each execute() gets its own fresh 10-email limit.

---

125. What are the DML-specific Limits?

Limit	Value
DML statements per transaction	150
Records processed by DML per transaction	10,000
Records in a single List for DML	10,000

DML statements — each `insert`, `update`, `delete`, `undelete` counts as 1 statement regardless of how many records are in the list.

```apex
// This is 1 DML statement (even with 200 records):
insert caseList;  // 1 DML, 200 records processed

// This is 200 DML statements (WRONG):
for (Case c : caseList) {
    insert c;  // 200 DML statements!
}
```

10,000 record limit: If you try to insert/update/delete more than 10,000 records in a single transaction, it fails. Use Batch Apex to process larger volumes.

---

126. What is the difference between LimitException and other Exceptions?

LimitException (Governor Limit violation):

- CANNOT be caught by try-catch
- Immediately kills the entire transaction
- All DML is rolled back
- Salesforce intentionally prevents catching this — forces developers to fix the root cause
- Examples: Too many SOQL queries, CPU time exceeded, Heap size exceeded

Other Exceptions (DmlException, NullPointerException, etc.):

- CAN be caught by try-catch
- You can handle them gracefully and continue execution
- The transaction doesn't necessarily roll back (depends on your handling)

```apex
// This WILL NOT work — LimitException is uncatchable:
try {
    // code that hits 101 SOQL queries
} catch (System.LimitException e) {
    // This catch block NEVER executes — transaction already dead
}

// This WILL work — DmlException is catchable:
try {
    insert duplicateRecord;
} catch (DmlException e) {
    System.debug('Duplicate caught: ' + e.getMessage());
    // Transaction continues — you can handle it
}
```

---

127. How do you avoid Governor Limits — Best Practices Summary?

SOQL Best Practices:

- Never put SOQL inside a loop — collect IDs, query once, use a Map
- Use selective filters on indexed fields
- Use LIMIT clause to cap results
- Query only the fields you actually need
- Use SOQL for-loops for large result sets
- Use Batch Apex for 50,000+ records

DML Best Practices:

- Never put DML inside a loop — collect into a List, one DML after the loop
- Use Database.insert(list, false) for partial success when appropriate
- Minimize the number of DML operations by combining records into fewer lists

CPU Best Practices:

- Avoid nested loops — use Maps for O(1) lookups instead of inner loops
- Move heavy processing to async (Batch, Queueable)
- Use efficient collections (Sets for uniqueness, Maps for lookup)

Heap Best Practices:

- Query only needed fields
- Use SOQL for-loops to process in chunks
- Nullify large variables after use
- Avoid string concatenation in loops — use List and String.join()

General Patterns:

- One Trigger Per Object — keeps logic organized and prevents compounding limits
- Bulkify everything — write all code assuming 200 records at once
- Use async for large/complex operations
- Monitor limits using Limits class during development

---

128. How do you debug Governor Limit issues?

Method 1 — Debug Logs:

- Setup → Debug Logs → Add user → Reproduce the action
- Look for "LIMIT_USAGE_FOR_NS" in the log — shows all limits consumed
- Search for "Number of SOQL queries: X out of 100"

Method 2 — Developer Console:

- Open Developer Console → Reproduce the action
- Click on the log → check "Execution Overview" panel
- "Limits" tab shows exactly how many queries, DML, CPU time were used

Method 3 — Limits class in code:

```apex
System.debug('=== LIMIT CHECK ===');
System.debug('SOQL: ' + Limits.getQueries() + '/' + Limits.getLimitQueries());
System.debug('DML: ' + Limits.getDmlStatements() + '/' + Limits.getLimitDmlStatements(
System.debug('CPU: ' + Limits.getCpuTime() + '/' + Limits.getLimitCpuTime());
System.debug('Heap: ' + Limits.getHeapSize() + '/' + Limits.getLimitHeapSize());
```

Place these debug statements before and after your critical logic blocks to identify which section is consuming the most resources.

Method 4 — Salesforce Optimizer:

- Setup → Salesforce Optimizer → Run to identify inefficient code, unused fields, and limit-heavy processes across the entire org.

---

129. What are the limits for Flows and Process Builder?

Flows and Process Builder share the same transaction limits as Apex:

Limit	Value
SOQL queries per transaction	100 (shared with Apex triggers in same transaction)
DML statements per transaction	150 (shared)
Records retrieved	50,000 (shared)
Processed interview elements per flow	2,000
Flows per transaction	No hard limit, but they share the same 100 SOQL / 150 DML pool

Key danger: If a record save fires a before trigger (uses 30 SOQL), then an after trigger (uses 40 SOQL), then a Record-Triggered Flow (uses 25 SOQL), and then a Workflow rule (uses 10

SOQL) — that's 105 SOQL in one transaction → LimitException.

This is why understanding the Order of Execution matters — everything in a single save transaction shares limits.

---

130. Quick Revision — The 10 Limits You MUST Memorize

#	Limit	Value	What happens if exceeded
1	SOQL queries	100 (sync)	LimitException — uncatchable, full rollback
2	DML statements	150	LimitException — uncatchable, full rollback
3	DML records	10,000	LimitException — uncatchable, full rollback
4	SOQL rows returned	50,000	LimitException — uncatchable, full rollback
5	CPU time	10,000 ms (sync)	LimitException — uncatchable, full rollback
6	Heap size	6 MB (sync)	LimitException — uncatchable, full rollback
7	Callouts	100 per transaction	LimitException — uncatchable, full rollback
8	@future calls	50 per transaction	LimitException — uncatchable, full rollback
9	Batch QueryLocator	50 million records	Governor limit for Batch start()
10	Batch execute size	200 default, 2000 max	Configurable in Database.executeBatch()

That's 20 Governor Limits questions (Q111–Q130). Your total is now **130 questions** across 8 parts. The limits most likely to be asked in your interview: **why limits exist (multi-tenant), the big 6 numbers (100/150/10000/50000/10s/6MB), SOQL-in-loop fix, LimitException being uncatchable, and how Batch Apex resets limits per chunk.** These 5 points cover 90% of governor limit interview questions.